Embodied Agents Take Control: Minimal-Interface Zero-Shot Agents Rival Industrial-Scale Policies in Vision-and-Language Navigation

Jian Zhou[1*] **Xunyi Zhao**[1,2*] **Gengze Zhou**[1] **Zerui Li**[1]
Sihao Lin[1,2] **Jiajun Liu**[2,3,4] **Qi Wu**[1,2]

[1]Australian Institute for Machine Learning, Adelaide University
[2]Responsible AI Research Centre [3]CSIRO Data61 [4]The University of Queensland
[*]Equal contribution

Abstract: Autonomous embodied agents must sustain a long decision-making loop that involves perceiving, acting, verifying, and self-correcting over many steps. Current systems sustain this loop through task-specific workflows or embodied policies. We study a third form, *agentic embodied control*, in which a general-purpose agent holds the loop itself. Using zero-shot navigation as a controlled testbed, we evaluate three software-engineering agent harnesses given only a monocular RGB camera and discrete actions. Under this strictly minimal condition, replicated default-effort configurations reach $70.7 \pm 3.5\%$ success (`opus-5`, mean over three runs), and `fable-5` reaches **78%** at maximum effort. When a trained waypoint tool is exposed alongside primitives as an optional capability, the hybrid `fable-5` agent reaches $76.7 \pm 0.6\%$ at default effort, using half the environment steps and less than one quarter of the wall time of the maximum-effort primitive run. Controlled interventions show that capability is primarily model-centered: model choice strongly changes success, harness effects are descriptive, and a forced waypoint interface helps weaker models but can hinder stronger ones. Performance nevertheless falls sharply on longer-horizon tasks, while latency and context growth limit sustained operation. These results show that agentic control is already competitive in zero-shot navigation and that models, harnesses, and interfaces offer complementary paths toward autonomous embodied agents.

Keywords: Embodied agents, zero-shot navigation, vision-language models

1 Introduction

Consider a household robot expected to operate autonomously day after day. It must find the kitchen, pick up a cup, and recover from mistakes without human rescue. Navigation and manipulation require different low-level controllers, but share the high-level challenge of reasoning over extended interaction. We ultimately seek an *Autonomous Embodied Agent* that can operate over extended interaction without repeated human rescue while managing its state, errors, and computational resources.

Two streams of work have dominated progress toward this goal. The 'trained' stream internalizes navigation capability in a policy that maps observations to actions, as exemplified by NaVid and its successors [1, 2]. The 'zero-shot' stream keeps a general-purpose model frozen and supplies the missing navigation structure outside its weights. NavGPT and MapGPT, for example, place an LLM inside fixed workflows for spatial representation and planning [3, 4]. The first stream puts capability in the weights, whereas the other assembles it through scaffolding. In both, an external environment loop or human-authored workflow determines what happens next (fig. 1, left).

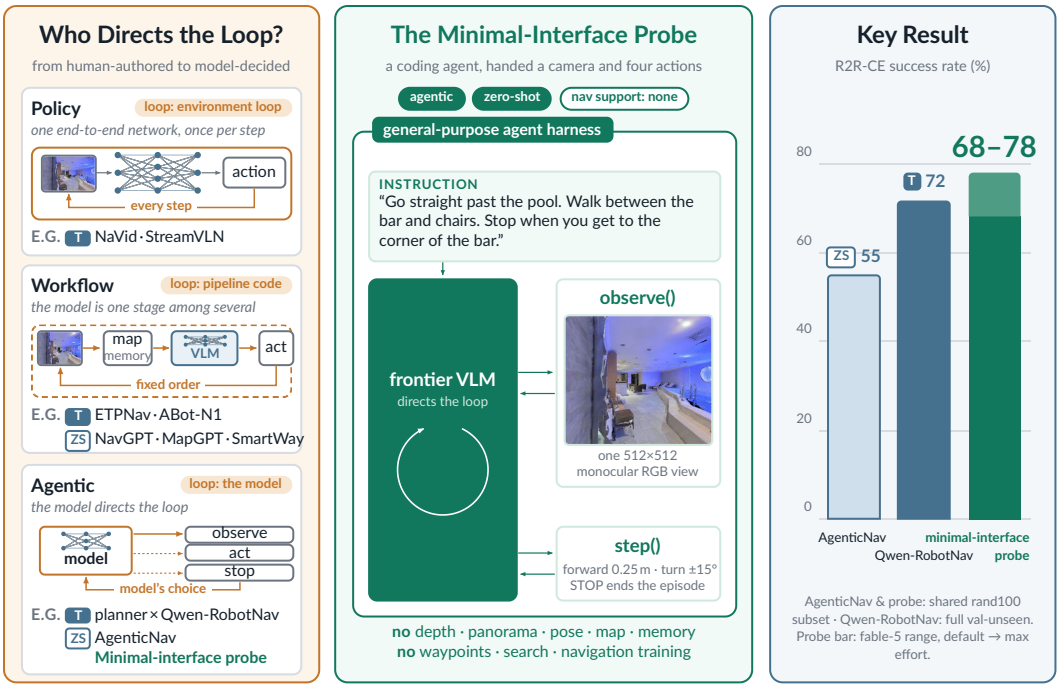


Figure 1: **Who holds the interaction loop, what we hand the model, and how far that gets it.** *Left:* at inference the loop is directed by an external environment loop (*policy*), human-authored pipeline code (*workflow*), or the model itself (*agentic*), with systems placed by the loop they execute, trained (T) or zero-shot (ZS). *Middle:* our minimal-interface probe, a software-engineering harness with one monocular RGB view (`observe()`) and four discrete primitives (`step()`), sits in the agentic zero-shot cell with no navigation-specific scaffolding (section 3.2). *Right:* R2R-CE success rate against the strongest prior zero-shot system (AgenticNav, same `rand100` subset) and the strongest trained navigator (Qwen-RobotNav, full val-unseen), the probe bar spanning its `fable-5` default-to-maximum-effort range (table 1).

More recently, industrial systems have combined the two ideas through slow–fast or "dual-brain" architectures. ABot-N1 and InternVLA-N1 couple a slow reasoning model to a trained fast action expert [5, 6]. These systems are more capable and hierarchical, but their handoff remains fixed. The slow stage emits a subgoal and the fast stage executes it. They are therefore trained workflows, not agentic systems. Architecture has deepened, but control of the interaction loop still lies in human-authored code.

This organization now deserves reconsideration. Frontier vision–language models increasingly combine perception across turns, spatial reasoning, progress tracking, and long-horizon tool use. Agents developed for software engineering further show that models can sustain extended tool-mediated interaction through generic loops [7, 8, 9]. Connecting successive observations, tracking progress, reasoning about action outcomes, and recovering from errors are also central to embodied behavior. Functions previously supplied by navigation-specific scaffolding may therefore be increasingly available in the model itself. This raises a broader question: *can the model take control of the embodied interaction loop?*

We study this question through vision-and-language navigation [10] because navigation requires language grounding, spatial reasoning, progress tracking, recovery, and stopping over extended interaction, while discrete actions reduce the confounding demands of low-level control. We equip three off-the-shelf agent harnesses originally built for software engineering with a monocular RGB camera and four discrete actions (fig. 1, middle). They receive no navigation-specific training, policy, map, search, or explicit memory. Unlike a policy, the model decides when to observe, which actions to issue, how to recover, and when to stop. Unlike a workflow, these decisions are not prescribed by

task-specific control code. We call this organization *agentic embodied control*. Using non-embodied harnesses ensures that the surrounding interaction logic was not engineered for navigation.

The answer is surprisingly strong. Under the strictly minimal interface, replicated default-effort runs average $70.7 \pm 3.5\%$ SR with `opus-5` and $68.3 \pm 1.5\%$ with `fable-5`, and `fable-5` reaches **78% SR** at maximum effort. On the same model and harness, an optional hybrid interface averages $76.7 \pm 0.6\%$ and nearly matches that peak with half the environment steps and less than one quarter of the wall time. These results use the standard zero-shot R2R-CE test set [11] inherited from Open-Nav and since adopted by SmartWay, AgenticNav, and other zero-shot systems [12, 13, 14]. In context, the minimal result is competitive with engineered zero-shot workflows and recent industrial navigators trained on millions of samples, although the trained results use a different split [15, 5].

Where, then, does the capability reside? With the harness and interface held fixed, model choice changes SR dramatically. With the model held fixed, harness differences show no consistent direction and, given run-to-run variance and unmatched serving paths, are descriptive rather than causal. A forced waypoint interface rescues weaker models but can constrain stronger ones; exposing it as an optional tool instead lets the agent compose coarse and fine control. Together, these results point to a *model-centered*, but not model-only, organization: the model supplies most of the decision capability, the harness sustains interaction, and the interface shapes how that capability is expressed. The relevant question is therefore not simply how much scaffolding to add, but whether it is imposed or placed under agent control.

Strong R2R-CE performance does not amount to general embodied competence. Success falls to 26–39% on the longer-horizon RxR-CE benchmark, while latency and unbounded context growth make sustained operation impractical. Navigation therefore reveals both a transition and a gap. Agentic systems can already produce capable embodied decisions, but cannot yet sustain the open-ended, resource-bounded autonomy required of an Autonomous Embodied Agent.

Our contributions are threefold:

- We instantiate agentic embodied control in navigation: unmodified software-engineering harnesses, one monocular RGB view, and four discrete primitives reach $70.7 \pm 3.5\%$ SR replicated and 78% at maximum effort on the standard zero-shot R2R-CE test set, without navigation-specific training, scaffolding, or workflows.
- Through single-axis interventions we locate the capability: model choice spans 5–72 SR while harness effects stay within a few points, and a trained waypoint module that rescues weak models when imposed makes the strongest agent better, faster, and cheaper when offered as an optional tool.
- We characterize what still separates this from sustained autonomy: success drops to 26–39 SR on longer horizons as context grows unboundedly, failures end in silent, confident claims of success, and deployment on a physical quadruped shows reasoning transfers while body awareness and spatial memory do not.

2 Related Work

2.1 Policies and Workflows in Embodied AI

VLN provides a controlled setting for this study. R2R introduced instruction- guided navigation on environment graphs [10], and R2R-CE extended it to continuous environments [11]. The trained line primarily develops navigation policies in which a model maps observations and history to an action inside an external environment loop. NaVid established a video-based VLM policy, followed by systems with streaming memory, larger models, and substantially more navigation data [1, 16, 2, 6, 15]. Vision–language–action models follow the same policy-centered organization in manipulation [17, 18, 19, 20].

The zero-shot line takes the opposite route by freezing a general LLM or VLM and constructs navigation capability through a workflow. NavGPT and MapGPT initiated this direction with fixed

prompting loops and external spatial representations [3, 4]. Later systems add panoramas, depth, maps, and waypoints [12, 13], structured planning, discussion, and search [21, 22], or memory, backtracking, collision handling, and verification [23, 24, 25, 26]. The model supplies semantic reasoning, but human-authored code fixes how these components are invoked.

A newer trained line combines reasoning and action through hierarchical workflows. ETPNav connects learned waypoint prediction and a cross-modal policy to topological planning [27]. More recent industrial slow–fast systems couple a deliberative model to a trained action expert. ABot-N1, InternVLA-N1, and Vesta differ in their components and schedules, but all execute a fixed planner/subgoal-to-controller handoff [5, 6, 28]. Their hierarchy is dynamic in content, but fixed in control flow.

These developments expose two orthogonal questions. Navigation capability may come from task-specific training or from frozen-model orchestration, while control may reside in a policy loop, a fixed workflow, or the model itself. We focus on the last case. Rather than adding another workflow [29], we remove most navigation-specific scaffolding and give the model authority over observation, action, recovery, and termination.

2.2 Agentic Control and Embodied Interfaces

ReAct [7] organizes an agent as a repeated observe–reason–act loop over interaction history. Coding agents show that capable models can sustain extended work through small sets of generic tools [8, 9, 30]. Embodied agents provide a related precedent. Voyager combines an LLM, environment feedback, and executable tools with a task-specific curriculum, skill library, and verification loop [31]. We ask whether a generic harness can support embodied behavior without such task-specific support.

Agentic control remains rare in embodied navigation. AgenticNav exposes actions, depth, and memory as tools selected by a frozen model [14], while the deployed Qwen-RobotNav system lets an upper-level planner repeatedly invoke a trained navigation policy and switch task modes [15]. These examples show that agentic control can orchestrate either zero-shot tools or trained policies. Our minimal setting isolates the former; the hybrid study then tests whether a trained module becomes more useful when placed under agent control.

We call a system *agentic* when the model, rather than an external environment loop or human-authored control graph, directs the high-level interaction. It chooses when to gather information, which actions to take, whether to revise its approach, and when to terminate. This concerns control authority, not merely the presence of an LLM or VLM. The harness still maintains the session and executes tools, while the interface bounds what the model can observe and express. Agent–computer interface design shows that this boundary can substantially shape behavior [9]. Accordingly, we separate the *model*, *harness*, and *interface*, three factors often changed together in embodied systems. Our models and harnesses are *general-purpose* only in provenance because none was developed for navigation. Their importance here is to expose agentic control without hiding navigation logic in the surrounding system.

3 Agentic Embodied Control through a Minimal Interface

3.1 Model-Directed Interaction

We replace the navigation scaffolding that zero-shot systems build around a frozen VLM with a general-purpose agent harness. Given the instruction and interaction history, the model decides when to observe, how to move, and when to stop. The harness delivers prompts, executes tool calls, returns results, and maintains the session, but does not prescribe the sequence of interaction. No navigation-specific module intervenes between model and environment. The model therefore holds high-level control authority rather than serving as a policy queried at every environment step or as a component in a fixed workflow.

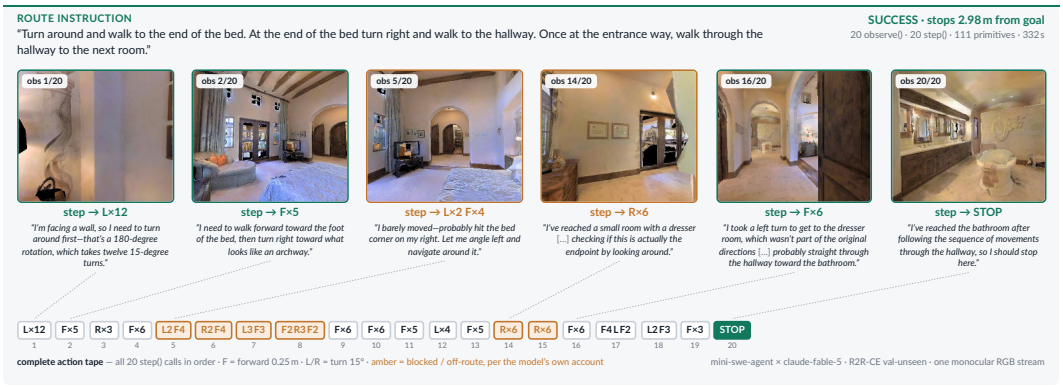


Figure 2: **One successful R2R-CE episode, reconstructed from its raw log.** Six of 20 archived observations, each with the `step()` call issued from that view and the recorded reasoning verbatim ([. . .] = elision). The bottom tape lists all 111 primitives in order (F = 0.25 m forward, L/R = 15° turn). Amber marks segments the model itself describes as blocked or off-route. No map, pose, depth, panorama, or waypoint is exposed (section 3.2). mini-swe-agent, `fable-5`, default effort.

The agent receives no navigation training or in-domain demonstrations. Every episode starts in a fresh session, with no state or experience carried across episodes. There is no external navigation memory, map builder, state estimator, planner, search procedure, learned policy, or verification stage. The only history is that maintained by the native harness. The setup is thus a diagnostic realization of agentic control, not a new navigation architecture.

3.2 A Minimal Perception–Action Interface

The embodied interface only contains two tools. `observe()` returns a single 512×512 front-facing RGB frame without advancing the environment. `step(actions)` executes an ordered sequence drawn from four Habitat primitives [32], consisting of `FORWARD`, `LEFT`, `RIGHT`, and `STOP`. Forward motion advances 0.25 m, each turn rotates 15°, and `STOP` terminates the episode. A sequence is limited only by the remaining 500-step budget. The tool reports the number of executed primitives and remaining budget, but no image or collision signal. The model must call `observe()` to see an action's effect.

These tools and the natural-language instruction form the entire task-specific interface. The agent receives no pose, odometry, depth, panorama, map, waypoint candidates, collision feedback, or privileged simulator state. It must infer landmarks, progress, revisitation, action effects, and the stopping point from its monocular observation history. This deliberately diagnostic interface serves as the controlled baseline for the fixed and optional waypoint interfaces in sections 4.3 and 4.4.

4 Experimental Evaluation

We first establish how far the minimal interface reaches (section 4.2), then ablate the model, harness, and interface (section 4.3), and test optional interface composition (section 4.4). We close with long-horizon limits, physical deployment, and failure modes.

4.1 Experimental Setup

We evaluate on the fixed `rand100` subset of R2R-CE val-unseen (n=100), shared with prior zero-shot systems [12, 13, 14]. In the minimal-interface condition, the agent receives only a 512×512 front-facing RGB frame and the four primitive actions in section 3.2, with no pose, depth, panorama, map, waypoint, or collision feedback. All settings for each cell are fixed in the run registry before evaluation. We report Habitat-native success rate (SR), success weighted by path length (SPL),

Table 1: **Minimal-interface performance in the context of recent embodied models and navigation approaches evaluated on R2R-CE.** Control and source follow the loop each system executes on its reported benchmark. Visual input: M = monocular, P = panorama, D = depth. Zero-shot evaluation follows section 4.1. Trained rows report full val-unseen results and are included for context. Ours in **bold**. Best external SR/SPL per block underlined. * denotes a mean over three independent runs. SPL for those rows is likewise a replication mean.

System (model / policy)	Control	Source	Visual	Nav. support	SR↑	SPL↑
NaVid [1]	policy	trained	M	explicit video memory	37	35.00
NaVILA [33]	policy	trained	M	VLA + RL gait	54	49.00
StreamVLN [2]	policy	trained	M	slow-fast cache	57	51.90
Hy-Embodied-VLM (A3B) [34]	policy	trained	M	MoE, map-free	58	54.20
NavFoM [16]	policy	trained	P	cross-embodiment	62	55.30
Qwen-RobotNav (Qwen3-VL) [15]	policy	trained	M	explicit memory, waypoint, planning	66	59.60
OmniNav [35]	policy	trained	M	flow-matching head	70	66.10
Qwen-RobotNav (Qwen3-VL) [15]	policy	trained	P	explicit memory, waypoint, planning	72	66.60
SmartWay (GPT-5.5) [13]	workflow	zero-shot	P+D	explicit memory, waypoint, search	44	35.04
Vesta [28]	workflow	trained	M	planner + ext. controller	56	50.80
ETPNav [27]	workflow	trained	P+D	map, waypoint, planning	57	49.15
InternVLA-N1 / DualVLN [6]	workflow	trained	M	dual-system, diffusion	64	58.50
ABot-N1 [5]	workflow	trained	3-cam	dual-brain, pixel goal	71	67.50
AgenticNav (GPT-5.5) [14]	agentic	zero-shot	P+D	map, explicit memory, search, action tools	55	48.41
Minimal (fable-5, mini-swe-agent)	**agentic**	**zero-shot**	**M**	**none**	**72**	**59.08**
Minimal (fable-5, Claude Agent SDK [36])	**agentic**	**zero-shot**	**M**	**none**	***68.3**	**58.02**
Minimal (opus-5, Claude Agent SDK)	**agentic**	**zero-shot**	**M**	**none**	***70.7**	**55.21**
Minimal (fable-5, Claude Agent SDK [36], max effort)	**agentic**	**zero-shot**	**M**	**none**	**78**	**65.27**

navigation error (NE), and oracle success rate (OSR). Full protocols and configuration details are provided in section A.

Most cells contain one run over the same 100 episodes due to evaluation cost. For the default-effort Claude SDK cells, we conduct three independent runs and report their mean and sample standard deviation (table 2). A leading star (*) marks such a mean throughout. Across these cells, the within-configuration standard deviation ranges from 1.2 to 3.5 SR points, providing a rough empirical scale for run-to-run variation. As this study uses the minimal interface as a descriptive probe, we base our conclusions on broad patterns and substantial contrasts rather than differences of only a few SR points. For example, waypoints improve the Qwen models by 19–38 SR points.

4.2 Main Results with a Minimal Embodied Interface

How capable is an agent given only the minimal interface? Table 1 compares four representative configurations from our results board (table 2) with recent embodied models and navigation approaches evaluated on R2R-CE. We include the standard mini-swe-agent configuration, two replicated Claude SDK configurations at default effort, and our strongest configuration. The zero-shot rows follow section 4.1. Trained rows use the full val-unseen split, so they serve only as a reference for the performance range.

Our four frontier-model configurations reach 68.3–78 SR. On the same zero-shot subset, AgenticNav reaches 55 SR with a map, explicit memory, search, and additional action tools. The comparison is not controlled because the systems use different models and serving paths. Even so, the results show that navigation-specific scaffolding is not necessary for strong zero-shot performance. With only the minimal interface, frontier models already reach the performance range of recent industrial-scale trained policies.

Table 2 presents the full main board. Performance varies widely even when the minimal interface is held fixed, showing that the interface itself does not provide the navigation capability. Instead, it serves as a probe of the embodied control available in the underlying model. We examine this variation along the model, harness, and interface axes next (section 4.3).

4.3 Ablations of the Model, Harness, and Interface

Each ablation varies one contributor while holding the others at the frozen baseline of section 4.1. The harness axis realizes the same interface through three harnesses built for software engineering:

Table 2: **The main minimal-interface board.** Main-board cells use the frozen protocol (section 4.1) at vendor-default reasoning effort (section A.4). The last row reports `fable-5` at maximum effort, the strongest configuration in table 1. Other effort variants are in section 4.3. Claude SDK cells at default effort report mean ± sample s.d. over three clean replications.

Harness	Model	SR↑	SPL↑	NE↓	OSR↑
mini-swe-agent	`qwen3.5-4b`	5	4.58	8.93	11
	`qwen3.5-9b`	7	5.36	8.63	15
	`qwen3.5-plus`	34	26.74	6.32	48
	`qwen3.7-plus`	42	32.85	6.81	55
	`qwen3.6-plus`	45	33.27	6.25	57
	`gpt-5.5`	52	44.24	7.29	57
	`gpt-5.6`	60	42.04	4.99	68
	`sonnet-5`	53	38.14	5.52	61
	`opus-4.8`	63	52.77	4.21	65
	`fable-5`	72	59.08	4.48	77
	`opus-5`	69	50.24	5.15	78
Claude SDK	`sonnet-5`	$^*51.3 \pm 1.2$	37.84 ± 0.89	5.80 ± 0.43	61.3 ± 1.2
	`opus-4.8`	$^*55.7 \pm 2.3$	47.31 ± 3.81	5.24 ± 0.44	59.3 ± 0.6
	`fable-5`	$^*68.3 \pm 1.5$	58.02 ± 1.50	5.13 ± 0.26	73.3 ± 1.5
	`opus-5`	$^*70.7 \pm 3.5$	55.21 ± 2.73	4.79 ± 0.43	78.3 ± 5.7
Codex CLI	`gpt-5.5`	45	35.74	5.66	51
	`gpt-5.6`	56	41.57	6.15	64
Claude SDK	`fable-5` (max effort)	78	65.27	3.84	83

Table 3: **Reasoning-effort ablation.** Effort labels are vendor-specific. *: replication mean (table 2). Time/ep: mean and median wall-clock seconds per episode (default cells pooled over the three replication runs). †Time averaged over episodes with logged wall time (eight episodes timed out).

Model	Harness	Effort change	SR	Δ	Mean t/ep (s)	Med. t/ep (s)
`sonnet-5`	Claude SDK	default → max	$^*51.3 \to 56$	+4.7	$446 \to 698$	$323 \to 532$
`opus-4.8`	Claude SDK	default → max	$^*55.7 \to 56$	+0.3	$437 \to 899^\dagger$	$254 \to 652^\dagger$
`fable-5`	Claude SDK	default → max	$^*68.3 \to 78$	+9.7	$379 \to 633$	$210 \to 484$
`opus-5`	Claude SDK	default → max	$^*70.7 \to 74$	+3.3	$332 \to 553$	$228 \to 429$
`gpt-5.5`	Codex CLI	default → xhigh	$45 \to 50$	+5	—	—
`gpt-5.6`	Codex CLI	default → xhigh	$56 \to 62$	+6	—	—
`gpt-5.5`	mini-swe-agent	default → xhigh	$52 \to 50$	−2	—	—
`gpt-5.6`	mini-swe-agent	default → xhigh	$60 \to 55$	−5	—	—

mini-swe-agent, the minimal reference harness used to compare language models on the SWE-bench bash-only leaderboard [30, 8]; the Claude Agent SDK [36]; and the Codex CLI, OpenAI's local software-agent harness [37]. None of them adds a navigation-specific planner, map, or policy.

Model. Unsurprisingly, stronger models navigate better under the minimal interface. With the loop and action space fixed, changing only the VLM spans 5–72 SR (fig. 3), the widest range in the study. This dependence is consistent with three general capabilities required by the interface: spatial scene understanding, sustained multi-turn multimodal interaction, and retrieval from a growing context. The long-horizon analysis further exposes limitations in retrieving information from context (section 4.5). These capabilities arise without navigation training, suggesting further headroom from embodiment-specific post-training (section 5). The within-model control below shows that reasoning effort does not explain the model range.

Reasoning effort within the model axis. Holding the model, harness, interface, and episodes fixed, we vary only the vendor-defined reasoning effort. Table 3 summarizes the resulting SR changes.

Table 4: **Interface augmentation is selective.** Primitive vs. waypoint under the same model, harness, and episodes. Primitive entries for the Claude models are the replication means of table 2. All other entries are single runs. CR = collision rate (native to VLNVerse). ‡GPT pairs run on Codex CLI; the `gpt-5.6-sol` bare cell uses the product-default low effort while its waypoint cell pins medium, and both GPT waypoint cells use codex-cli 0.145.0 (the bare board froze 0.142.0).

		SR↑			SPL↑		CR↓	
Benchmark	Model	Primitive	Waypoint	Δ	Primitive	Waypoint	Primitive	Waypoint
R2R-CE	`qwen3.5-4b`	5	43	**+38**	4.58	34.18	–	–
R2R-CE	`qwen3.5-9b`	7	44	**+37**	5.36	32.81	–	–
R2R-CE	`qwen3.5-plus`	34	53	**+19**	26.74	44.05	–	–
R2R-CE	`gpt-5.5`‡	45	67	**+22**	35.74	58.85	–	–
R2R-CE	`gpt-5.6-sol`‡	56	73	**+17**	41.57	63.98	–	–
R2R-CE	`sonnet-5`	*51.3	60	+8.7	37.84	45.83	–	–
R2R-CE	`opus-4.8`	*55.7	65	+9.3	47.31	54.23	–	–
R2R-CE	`fable-5`	*68.3	69	+0.7	58.02	59.15	–	–
R2R-CE	`opus-5`	*70.7	72	+1.3	55.21	60.52	–	–
VLNVerse	`sonnet-5`	78	72	−6	52.06	22.86	6	35
VLNVerse	`fable-5`	84	80	−4	62.47	42.57	7	27

Only `fable-5` moves clearly, gaining 9.7 SR at maximum effort. The remaining shifts span -5 to $+6$ with no consistent direction. Effort effects are therefore model-specific rather than uniform. Full metrics are in section A.5.

Harness. We compare the open mini-swe-agent with vendor harnesses across six shared model identifiers (fig. 3B). Five pairs differ by 1.7–7.3 SR, while `opus-5` reverses direction with a 1.7-point vendor advantage. Given the variance in section 4.1 and unmatched serving paths and product defaults, we treat the comparison as descriptive rather than causal. More importantly, none of these harnesses was designed to support the long-term deployment of an autonomous embodied agent. Our task-only configurations provide neither persistent cross-task state nor embodied recovery and safety mechanisms. Together with the context growth observed on longer runs (section 4.5), this motivates embodied-native harnesses for resource-bounded, sustained operation.

Interface. We compare primitive control with a waypoint interface on R2R-CE and VLNVerse [38], holding the model, loop, and episodes fixed within each pair (table 4). The waypoint arm replaces primitives with a choice among at most five candidates proposed by a trained depth-based waypoint predictor [13, 39]. Its effect depends sharply on primitive-control ability. On R2R-CE, the gain generally shrinks as primitive performance rises: 37–38 SR for the small Qwen models, 17–22 for `qwen3.5-plus` and the Codex GPT models, roughly 9 for `sonnet-5` and `opus-4.8`, and only 0.7–1.3 for the two strongest models. On VLNVerse, the direction reverses: SR falls by 4–6 points, SPL by 20–29 points, and collision rate rises four- to six-fold. The waypoint interface is therefore compensatory rather than uniformly beneficial: it supplies useful spatial priors when primitive grounding is weak, but can impose a mismatched action abstraction when direct control is already effective. This conclusion is specific to the tested waypoint predictor and does not imply that richer interfaces are generally harmful.

4.4 Hybrid Interface: Better, Faster, Cheaper

The interface ablation above forces the agent to use one action space throughout. We instead expose primitives and waypoints simultaneously, turning the trained waypoint module from a mandatory control path into an optional capability. We evaluate three independent runs of this hybrid interface with `fable-5` on the Claude SDK at default effort (table 5).

The agent consistently adopts a coarse-to-fine strategy that the prompt does not prescribe. It uses waypoints early to localize the route and cover distance, then switches to primitives near the target to refine its final position. Across episodes, it combines roughly eight waypoint moves with five to six

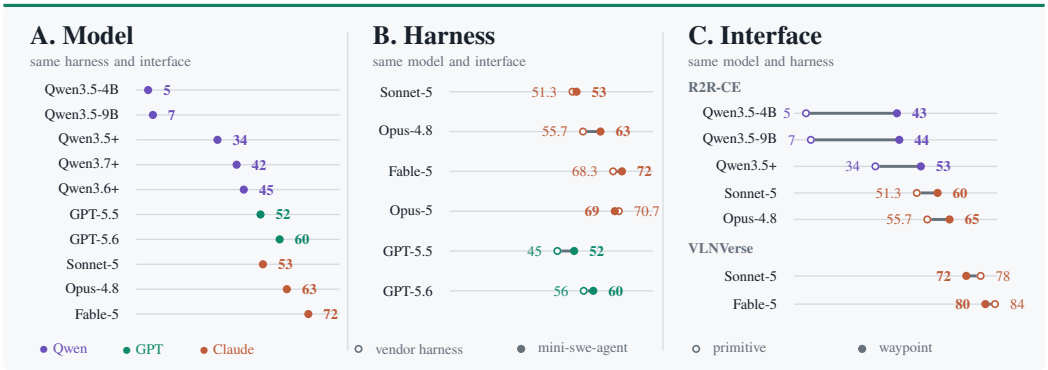


Figure 3: **Locating capability across model, harness, and interface** (shared SR scale). (A) With harness and interface fixed, the model spans 5–72 SR. (B) The open and vendor harness differ by 2–7 SR across the six shared pairs—mini-swe-agent leads in five, the vendor harness in one (descriptive; serving paths differ). (C) A waypoint interface rescues the weakest models on R2R-CE but reverses for strong models on VLNVerse, raising collision rate (CR).

Table 5: **Optional scaffolding improves success and efficiency.** (`fable-5`, Claude SDK, R2R-CE). *: mean ± s.d. over three runs. Steps, Time, Calls: per-episode medians of environment steps, wall-clock seconds, and LLM calls (default primitives and hybrid pooled over their three runs).

Interface	Effort	SR↑	SPL↑	NE↓	Med. steps	Med. time (s)	Med. calls
primitives	default	*68.3 ± 1.5	58.02 ± 1.50	5.13 ± 0.26	87	210	39
waypoint	default	69	59.15	4.30	38	91	15
hybrid	default	*76.7 ± 0.6	63.63 ± 1.67	3.49 ± 0.23	48	112	20
primitives	max	78	65.27	3.84	97	484	48

primitive bursts, and the three runs reach 76, 77, and 77 SR (76.7 ± 0.6). This exceeds both fixed interfaces at the same effort (68.3 for primitives and 69 for waypoints), while achieving the best NE among the `fable-5` cells. Relative to max-effort primitives, the hybrid nearly matches SR with half the environment steps and less than one quarter of the wall time. The result refines the fixed-interface ablation: scaffolding becomes beneficial when exposed as an optional capability rather than imposed as a control path. The agent, not the designer, decides when the trained module is worth invoking.

4.5 Long-Horizon Limitations

The strong R2R-CE result remains a *short-horizon* result. As the horizon lengthens, a deployment barrier and a capability ceiling emerge together, both associated with a rapidly growing interaction context (table 6).

Latency. The demonstrated capability is not real-time. Because the minimal loop re-sends its entire observation history on every call, context grows without bound within an episode: across the R2R-CE bare cells the final-turn context reaches a median of 33k tokens (maximum 169k), straining the request path on the longest runs. Across 4,700 logged simulator episodes, median wall time is 206 s, 18.6% exceed ten minutes, and 4.6% exceed twenty, totaling 467 hours. In the matched stress cells, moving from R2R-CE to RxR-CE doubles to triples the median final-turn context and at least doubles wall time. These measurements combine inference, deliberation, and harness overhead and do not isolate a causal component, but they expose the need for bounded context management.

Horizon. Under the same model, harness, effort, and budgets, primitive SR falls from 75 on R2R-CE to 26 on RxR-CE [40]. Waypoints improve RxR-CE to 39 but remain 30 points below their R2R-CE result. The longer, more winding routes accumulate drift, backtracking, and views less tightly aligned with the instruction; R2R-CE histories, by contrast, remain almost pure signal.

Table 6: **Long-horizon stress** under the same frozen configuration (`fable-5`, Claude SDK, default effort; 100-episode English random samples, identical budgets). Context: tokens entering the final LLM call of an episode.

	Interface	R2R-CE	RxR-CE
SR↑	primitives	75	26
	waypoint	69	39
wall time (median, s)	primitives	255	527
	waypoint	91	336
final context (median, tokens)	primitives	22.6k	49.2k
	waypoint	14.6k	44.2k

We hypothesize that this growing history dilutes task-relevant evidence; capping or windowing the context is required to test that mechanism causally.

Together, these results mark the gap between short-episode agentic control and an autonomous embodied agent: sustained operation requires an embodied harness that selectively retains and consolidates state, together with training that teaches the model to use that structure (section 5).

4.6 Simulator Behavior/Failure Analysis

Of the 100 episodes in one complete `fable-5`–Claude SDK default-effort run (SR 75), 25 fail. We reconstructed each failure from its raw log and audited every diagnosis frame by frame against the archived observations (18 confirmed, 7 refined in mechanism, none refuted). Table 7 groups them into four mutually exclusive categories, and section B analyses all 25 with one representative episode per category in the format of fig. 2.

Table 7: **Failure taxonomy of the analyzed run** (25 failed episodes). med. d_{goal}: median final distance to goal (m). OSR: entered the 3 m success region at some point. "success": closing message claims the goal was reached.

failure category	n	med. d_{goal}	OSR	"success"
A wrong referent / branch	12	15.3	0	12
B stop decision	6	4.4	4	6
C runaway search	4	20.1	0	2
D geometry / simulator	3	9.2	0	2

Failure is **silent**: 23 of 25 episodes end in a voluntary STOP, 22 under an explicit claim of success, and stated confidence is uncorrelated with the final distance to goal, which ranges from 3.1 to 44.6 m under near-identical "well within 3 meters" conclusions. It is also **route-level, not stop-level**: only 4 of 25 trajectories ever entered the 3 m success region, so the typical failure diverged early and never returned.

The mechanisms are linguistic and spatial rather than perceptual. The largest category binds an ambiguous referent ("the sink", "the first bed") to the wrong instance in a scene full of duplicates, and stop-decision failures anchor on the named object rather than the annotated endpoint, though range estimates are usually accurate. The signature cognitive failure is *self-diagnosis without self-correction*: in at least 10 episodes the reasoning states the correct doubt verbatim and the agent commits to the wrong endpoint anyway. Geometry matters only where the interface hides it, since `step()` returns no collision or pose signal and a blocked forward still reports `executed == requested`, so wedges, mirror reflections, and reconstruction holes must be read from pixels alone, and in three episodes are not.

4.7 Deployment on a Physical Robot

We deploy the same minimal agent on a real Unitree Go2 quadruped in an office building, with the same single monocular view and primitive `step` actions as in the simulator; the robot's low-level control is wrapped behind this interface. In early trials the agent completed simple landmark-following instructions almost every time (the main exception is the body-clearance issue described below), so plain navigation tells us little. The 31 episodes across two sessions instead probe what a home robot would face—conditional reasoning, counting, multistage routes, fetching an object and returning, and handing an object to a specific person. Hardware gives no ground-truth pose, so each episode is read from its frames and the motion `step` reports. Section C gives the figures and analysis for each episode.

Reasoning and perception carry over. The agent settles a condition—logical or perceptual, looking first when it must—before taking the branch it selects; tells people apart by a fine attribute, delivering to the person in white shoes rather than a bystander; holds state across multi-leg routes such as fetch-and-return; and notices from the reported motion when a turn falls short, correcting at the next look.

Embodiment does not. The agent barely senses its own body: leaving a doorway, it turns before the body trailing half a meter behind the lens has cleared, and wedges. Actuation is coarse and `step` reports every request as executed, so a turn that falls short silently sends the rest of the instruction down the wrong heading. Most limiting, it keeps no persistent map, so tasks that integrate views over time break down—counting objects across revisits, picking among similar targets, judging distance. This is the embodied form of the long-horizon limit (section 4.5): the bottleneck moves off language and onto body awareness, calibrated motion, and a memory of space, which is where the next section is aimed.

5 Discussion: A Bootstrapping Cycle toward Autonomous Embodied Agents

Taken together, our results expose the gap between episode-scale agentic control and an autonomous embodied agent, while suggesting how the model, harness, and interface may jointly close it.

Agentic as a compositional substrate. Policy, workflow, and agentic systems are not mutually exclusive forms. An agentic system can expose a trained policy as an action model or tool, and a human-authored workflow as a skill, while letting the agent decide when to invoke, combine, or stop them. The key change is not that policies and workflows disappear, but that they move from holding global control to becoming capabilities under the agent's control. Agentic systems therefore provide an inclusive substrate on which policies, workflows, maps, planners, and other domain logic can be organized as callable and composable capabilities. Our hybrid-interface cell (section 4.4) shows this substrate in miniature: offered both action spaces, the agent decides per step when the trained waypoint module is worth invoking, and surpasses either fixed interface.

Model, harness, and interface bootstrap one another. Our experiments identify a distinct development target in each layer: the model sweep and failure analysis expose limits in spatial reasoning and self-correction; the long-horizon results expose the need for persistent, resource-bounded state management; and the hybrid and real-robot studies show both the leverage and remaining limits of interfaces and action models that connect reasoning to physical control. Neighboring agent domains provide useful precedents: GUI agents have been trained from synthesized and iteratively refined interaction traces [41, 42], while coding agents have learned from software evolution and rule-based execution feedback [43]. We hypothesize that embodied agents could follow a similar cycle: maps, waypoint modules, action models, and engineered harnesses help produce successful and verifiable trajectories; those trajectories support supervised fine-tuning, reinforcement learning with grounded or verifiable rewards, and distillation [44, 7, 45, 46]; and stronger models in turn change or simplify the scaffolds they need. We do not test this cycle here, and which capabilities should be internalized, retained in the harness, or delegated through tools remains open.

6 Conclusion

We replaced the navigation scaffolding with a generic agent harness, one monocular RGB view, and four primitive actions, and found that this minimal-interface, zero-shot agent reaches 78% SR on the standard zero-shot R2R-CE test set, competitive in context with engineered zero-shot systems and the trained state of the art. Separating model, harness, and interface shows how much capability already comes from the general-purpose model, and how strongly its expression depends on the system around it. The most telling result is what happens when the designer stops choosing: a trained waypoint module that rescues weak models yet constrains strong ones as a forced control path makes the strongest agent better, faster, and cheaper as an optional tool. The same interface carries over to a physical robot; what remains open is deployment beyond the episode—persistent memory, bounded context, and recovery over hours of operation. Here each layer is a direction of progress, and we expect a bootstrapping cycle to emerge across them: scaffolds generating the experience that trains better models, and better models in turn simplifying the scaffolds they need.

References

[1] J. Zhang, K. Wang, R. Xu, G. Zhou, Y. Hong, X. Fang, Q. Wu, Z. Zhang, and H. Wang. Navid: Video-based vlm plans the next step for vision-and-language navigation. *arXiv preprint arXiv:2402.15852*, 2024.

[2] M. Wei, C. Wan, X. Yu, T. Wang, Y. Yang, X. Mao, C. Zhu, W. Cai, H. Wang, Y. Chen, et al. Streamvln: Streaming vision-and-language navigation via slowfast context modeling. *arXiv preprint arXiv:2507.05240*, 2025.

[3] G. Zhou, Y. Hong, and Q. Wu. Navgpt: Explicit reasoning in vision-and-language navigation with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 7641–7649, 2024.

[4] J. Chen, B. Lin, R. Xu, Z. Chai, X. Liang, and K.-Y. Wong. Mapgpt: Map-guided prompting with adaptive path planning for vision-and-language navigation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9796–9810, 2024.

[5] R. Gong, Y. Guo, J. Hu, J. Kong, X. Leng, T. Li, W. Li, F. Liu, Z. Liu, J. Lu, M. Luo, C. Ming, Y. Shen, J. Tao, Z. Wang, M. Yin, et al. ABot-N1: Toward a general visual language navigation foundation model, 2026. URL https://arxiv.org/abs/2607.10383.

[6] M. Wei, C. Wan, J. Peng, X. Yu, Y. Yang, D. Feng, W. Cai, C. Zhu, T. Wang, J. Pang, and X. Liu. Ground slow, move fast: A dual-system foundation model for generalizable vision-and-language navigation, 2025. URL https://arxiv.org/abs/2512.08186.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.

[8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[9] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

[10] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, and A. Van Den Hengel. Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3674–3683, 2018.

[11] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee. Beyond the nav-graph: Vision-and-language navigation in continuous environments. In *European Conference on Computer Vision*, pages 104–120. Springer, 2020.

[12] Y. Qiao, W. Lyu, H. Wang, Z. Wang, Z. Li, Y. Zhang, M. Tan, and Q. Wu. Open-nav: Exploring zero-shot vision-and-language navigation in continuous environment with open-source llms. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6710–6717. IEEE, 2025.

[13] X. Shi, Z. Li, W. Lyu, J. Xia, F. Dayoub, Y. Qiao, and Q. Wu. Smartway: Enhanced waypoint prediction and backtracking for zero-shot vision-and-language navigation. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 16923–16930. IEEE, 2025.

[14] Y. Li, C. Li, H. Shi, J. Luo, J. Cai, M. Yang, and T. Qin. AgenticNav: Zero-shot vision-and-language navigation as a tool-calling harness, 2026. URL https://arxiv.org/abs/2606.10577.

[15] J. Zhang, G. Zhou, H. Yin, Y. Huang, Z. Lei, Q. Peng, H. Yuan, J. Zhang, X. Guo, X. Chen, A. Yang, F. Huang, J. Lin, D. Liu, J. Zhou, Z. Yu, J. Fan, Z. Liang, P. Lin, Y. Wang, A. Chen, K. Yan, X. Xu, J. Li, L. Hu, M. Zhang, S. Li, W. Xiao, S. Bai, X. Ren, C. Lv, C. Wu, and X.-H. Chen. Qwen-RobotNav technical report: A scalable navigation model designed for an agentic navigation system, 2026. URL https://arxiv.org/abs/2606.18112.

[16] J. Zhang, A. Li, Y. Qi, M. Li, J. Liu, S. Wang, H. Liu, G. Zhou, Y. Wu, X. Li, Y. Fan, W. Li, Z. Chen, F. Gao, Q. Wu, Z. Zhang, and H. Wang. Embodied navigation foundation model, 2025. URL https://arxiv.org/abs/2509.12129.

[17] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence. PaLM-E: An embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488. PMLR, 2023. URL https://proceedings.mlr.press/v202/driess23a.html.

[18] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.

[19] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model, 2024.

[20] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control, 2024.

[21] Y. Long, X. Li, W. Cai, and H. Dong. Discuss before moving: Visual language navigation via multi-expert discussions. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 17380–17387. IEEE, 2024.

[22] Y. Long, W. Cai, H. Wang, G. Zhan, and H. Dong. Instructnav: Zero-shot system for generic instruction navigation in unexplored environment. In *Conference on Robot Learning*, pages 2049–2060. PMLR, 2025.

[23] D. Li, W. Chen, and X. Lin. TINA: Think, interaction, and action framework for zero-shot vision language navigation, 2024. URL https://arxiv.org/abs/2403.08833.

[24] S. Jeong, G.-C. Kang, J. Kim, and B.-T. Zhang. Zero-shot vision-and-language navigation with collision mitigation in continuous environment, 2024. URL https://arxiv.org/abs/2410.17267.

[25] H. Li, X. Dong, H. Jiang, Y. Zhou, and X. Ma. CMMR-VLN: Vision-and-language navigation via continual multimodal memory retrieval, 2026. URL https://arxiv.org/abs/2603.07997.

[26] Z. Li, S. Li, Z. Zhang, B. Li, and S. Zhou. DV-VLN: Dual verification for reliable LLM-based vision-and-language navigation, 2026. URL https://arxiv.org/abs/2601.18492.

[27] D. An, H. Wang, W. Wang, Z. Wang, Y. Huang, K. He, and L. Wang. Etpnav: Evolving topological planning for vision-language navigation in continuous environments. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[28] J. Bjorck, Z. Li, Y. Man, J. Wang, A.-C. Cheng, S. Liu, S. Wang, Z. Yu, A. Badki, S. Birchfield, V. Blukis, Y. Chebotar, S. Chen, S. Leng, Y.-C. Chou, T. Ding, B. Li, Z. Luo, H. Su, J. Tremblay, T. Wang, B. Wen, J. Wu, X. Xie, H. Ye, H. Yin, K. R. Zentner, L. Gui, Y.-X. Wang, Y. Zhu, L. Fan, and J. Kautz. Vesta: A generalist embodied reasoning model, 2026. URL https://arxiv.org/abs/2606.20905.

[29] J. Zhou, S. Lin, J. Li, S. Fu, G. Zhou, and Q. Wu. Automating the design of embodied agent architectures, 2026. URL https://arxiv.org/abs/2606.30111.

[30] The SWE-agent team. mini-swe-agent: The minimal AI software engineering agent. https://github.com/SWE-agent/mini-swe-agent, 2025.

[31] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.

[32] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, et al. Habitat: A platform for embodied ai research. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9339–9347, 2019.

[33] A.-C. Cheng, Y. Ji, Z. Yang, Z. Gongye, X. Zou, J. Kautz, E. Bıyık, H. Yin, S. Liu, and X. Wang. NaVILA: Legged robot vision-language-action model for navigation, 2024. URL https://arxiv.org/abs/2412.04453.

[34] Z. Wang, X. Yu, Y. Rao, Y. Ling, Y. Li, O. Wang, M. Gao, Y. Zhou, Y. Liang, Z. Liu, Y. Zhang, R. Huang, X. Xu, B. Yuan, Y. Yuan, X. Tan, H. Zhang, Y. Huang, S. Zhang, H. Wu, H. Hu, and Z. Zhang. Hy-Embodied-VLM-1.0: Efficient physical-world agents, 2026. URL https://arxiv.org/abs/2607.12894.

[35] X. Xue, J. Hu, M. Luo, S. Xie, J. Chen, Z. Xie, K. Quan, W. Guo, M. Xu, and Z. Chu. OmniNav: A unified framework for prospective exploration and visual-language navigation, 2025. URL https://arxiv.org/abs/2509.25687.

[36] Anthropic. Claude agent SDK. https://docs.anthropic.com/en/api/agent-sdk, 2025.

[37] OpenAI. Unrolling the Codex agent loop. https://openai.com/index/unrolling-the-codex-agent-loop/, 2026.

[38] S. Lin, Z. Li, X. Zhao, G. Zhou, L. Wang, R. Wei, R. Tang, J. Li, H. Wang, J. Pang, A. van den Hengel, J. Liu, and Q. Wu. VLNVerse: A benchmark for vision-language navigation with versatile, embodied, realistic simulation and evaluation, 2025. URL https://arxiv.org/abs/2512.19021.

[39] Y. Hong, Z. Wang, Q. Wu, and S. Gould. Bridging the gap between learning in discrete and continuous environments for vision-and-language navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15439–15449, 2022.

[40] A. Ku, P. Anderson, R. Patel, E. Ie, and J. Baldridge. Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4392–4412, 2020.

[41] Y. Xu, D. Lu, Z. Shen, J. Wang, Z. Wang, Y. Mao, C. Xiong, and T. Yu. Agenttrek: Agent trajectory synthesis via guiding replay with web tutorials. *arXiv preprint arXiv:2412.09605*, 2024.

[42] Y. Qin, Y. Ye, J. Fang, H. Wang, S. Liang, S. Tian, J. Zhang, J. Li, Y. Li, S. Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

[43] Y. Wei, O. Duchenne, J. Copet, Q. Carbonneaux, L. Zhang, D. Fried, G. Synnaeve, R. Singh, and S. I. Wang. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.

[44] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[45] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[46] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[47] A. Chang, A. Dai, T. Funkhouser, M. Halber, M. Niessner, M. Savva, S. Song, A. Zeng, and Y. Zhang. Matterport3D: Learning from RGB-D data in indoor environments. In *International Conference on 3D Vision*, pages 667–676, 2017. doi:10.1109/3DV.2017.00081.

[48] Ollama. Ollama documentation. https://docs.ollama.com/, 2026.

[49] Alibaba Cloud. DashScope documentation. https://help.aliyun.com/zh/model-studio/, 2026.

A Experimental Settings

This appendix records the complete experimental configuration. The authoritative source is the released run registry (configuration as code, section 4.1); this appendix is a faithful transcription of it.

A.1 Task, benchmark, and metrics

Benchmark. R2R-CE [11] val-unseen on Habitat-Sim 0.1.7 [32] with Matterport3D scenes [47] and R2R-CE v1-3 episode definitions (identical to the release except for the spawn-heading and precomputed instruction-token fields; headings are arbitrary in R2R-CE and agents receive the raw instruction text). We evaluate episodes 0–99 of the fixed `rand100` val-unseen sample (n=100) introduced by Open-Nav [12] and shared by SmartWay and AgenticNav [13, 14]. Episodes are placed by index by the driver. The agent never chooses or observes the episode identity.

Success criterion and metrics. An episode succeeds iff the agent *itself* issues `STOP` within 3 m geodesic distance of the goal. An episode that exhausts its budget without stopping scores zero from any position. All metrics (SR, SPL, NE, OSR) are the standard VLN-CE measures, read *driver-side* after the session ends. The agent can never observe its own score: the shortest-path and oracle sensors present in the raw observation dictionary reach the tool bridge but are discarded there, never forwarded.

A.2 Observation and action space

- `observe()` returns a single egocentric RGB frame at 512×512 px with an HFOV of 90° (the VLN-CE camera configuration). This pure read advances nothing.
- `step(actions)` takes a list of up to 50 discrete primitives executed in order. `0` = `STOP` (terminal), 1 = forward 0.25 m, 2/3 = turn left/right 15° (the standard VLN-CE action space). The return value reports how many primitives executed and the remaining action budget.

No pose, odometry, depth, panorama, waypoint candidates, map, or cross-episode memory is available in the minimal-interface condition. Every frame the agent ever saw is archived alongside the trajectory.

A.3 Frozen run configuration

Table 8 lists the values frozen in the run registry. A run is launched by naming a registered board cell; none of these values is accepted as a command-line parameter. Launching with any override demotes the run to an off-board name and permanently excludes it from the board (section 4.1). The single exemption is episode selection for re-runs, which is not treated as a deviation.

Table 8: Frozen configuration, identical for every board cell.

Knob	Value
Episode set	R2R-CE `rand100` val-unseen sample, indices 0–99 (n=100)
Render resolution	512×512 RGB, HFOV 90°
LLM-call limit	200 per episode (hard on two of the three harnesses)
Action budget	500 low-level simulator steps (Habitat's episode-step cap)
Episode timeout	2400 s, driver-side kill at +600 s (in-session enforcement on mini-swe-agent only)
System prompt	one frozen task briefing, single source file, byte-identical across harnesses
Sessions	one fresh session per episode, no state survives

The briefing itself is short. Rendered per episode with the instruction text and the action budget, it reads verbatim:

```
You are controlling a robot in a real indoor environment (a
photorealistic 3D scan of a building). You interact only through these
tools:
- observe(): look through the robot's forward-facing camera (returns an
RGB image).
- step(actions): execute movement actions in order. 0 = STOP (permanently
ends the episode — declares you have reached the goal), 1 = move forward
0.25 m, 2 = turn left 15 degrees, 3 = turn right 15 degrees.
Your task is to follow this navigation instruction to its endpoint:
"{instruction}"
Rules:
- Alternate observing and stepping: look, decide where the instruction
wants you to go next, move, look again.
- You have a budget of {budget} movement actions.
- You succeed only if you issue action 0 (STOP) while within 3 meters of
the instruction's endpoint. STOP is permanent — issue it only when you
believe you are at the goal.
```

```
- Turning in place (e.g. step([2,2,2,2,2,2])) is a cheap way to look
 around when unsure.
- Work autonomously until you stop; nobody can answer questions.
```

The fixed opening user message is "Begin navigating. Call observe() first to see where you are."

A.4 Reasoning (extended thinking) configuration

Extended reasoning is enabled on *every* board cell at the per-cell setting recorded in table 9. Every surface runs at its vendor *default* effort on the main board, and the elevated settings (SDK Claude `max`, GPT `xhigh`) are the ablation of section 4.3. Effort labels are vendor-defined and **not commensurable across vendors**. We therefore also log the realized reasoning volume (tokens, and characters where readable) of every LLM call, and report it alongside any cross-vendor comparison.

Table 9: Reasoning effort per board cell. *Default*: main-board setting. *elevated*: effort-ablation setting (section 4.3). Codex default is `medium` on `gpt-5.5` and `low` on `gpt-5.6`. *logged*: whether reasoning content is recorded, an API property we cannot equalize. `qwen3.5-4b/9b` are served locally via Ollama [48]. The `plus` models via DashScope [49].

Cells	Default	Elevated	Logged
SDK × Claude	adaptive	`max`	summaries
mini-swe-agent × Claude	adaptive	`max`	summaries
mini-swe-agent × GPT	`medium`	`xhigh`	token counts
mini-swe-agent × Qwen	on (default)	not swept	full text
Codex × GPT	`medium/low`	`xhigh`	summaries

On the Anthropic Messages API, effort is carried as `output_config.effort` under the beta header `effort-2025-11-24` (a top-level `effort` field is rejected), and thinking on *tool-result continuation turns* (which is every turn of the navigation loop) additionally requires the beta header `interleaved-thinking-2025-05-14`. On the OpenAI API side, `gpt-5.6` rejects tool-carrying requests at its *default* reasoning tier (HTTP 400), so every GPT cell pins an explicit effort level.

We verified that the configuration is causally effective: with it, the mini-swe-agent Claude cells produce per-episode reasoning volumes on par with the SDK cells on the same model (paired $n{=}100$ means, *e.g.* ∼10.0k vs. ∼11.1k characters on `sonnet-5` and ∼8.9k vs. ∼8.8k on `fable-5`). Without the continuation-turn beta, reasoning volume is exactly zero across the whole episode.

A.5 Reasoning-effort metrics

Full metrics for the within-model effort ablation of section 4.3; per-cell settings are in section A.4.

Table 10: **Full reasoning-effort results on R2R-CE.** Each variant is paired with the default-effort cell of the same model and harness. *: replication mean over three clean runs (table 2). All other entries are single runs. †: NE averaged over the 92 episodes with logged metrics; the eight timed-out episodes count as failures in SR, SPL, and OSR.

Harness	Model	Effort	SR↑	SPL↑	NE↓	OSR↑
Claude SDK	`sonnet-5`	default	*51.3	37.84	5.80	61.3
		max	56	44.97	5.39	65
Claude SDK	`opus-4.8`	default	*55.7	47.31	5.24	59.3
		max	56	49.53	4.45†	59
Claude SDK	`fable-5`	default	*68.3	58.02	5.13	73.3
		max	78	65.27	3.84	83
Claude SDK	`opus-5`	default	*70.7	55.21	4.79	78.3
		max	74	55.97	4.98	79
Codex CLI	`gpt-5.5`	default	45	35.74	5.66	51
		xhigh	50	41.33	5.46	53
Codex CLI	`gpt-5.6`	default	56	41.57	6.15	64
		xhigh	62	49.04	4.92	66
mini-swe-agent	`gpt-5.5`	default	52	44.24	7.29	57
		xhigh	50	42.61	5.51	53
mini-swe-agent	`gpt-5.6`	default	60	42.04	4.99	68
		xhigh	55	40.34	5.56	59

B Case Study: All 25 Failures

Of the 100 R2R-CE episodes in the analyzed run (claude-sdk × `fable-5`, default effort, SR 75), 25 fail. We reconstruct every failure from its raw log in two passes. A first pass diagnoses each episode from the text stream alone (the instruction, the model's summarized reasoning, the tool-call sequence, and the environment feedback) and assigns a primary failure mechanism. A second, independent pass then audits each diagnosis against the archived observations, frame by frame: 18 of the 25 diagnoses were confirmed, 7 were revised in mechanism (none in category), and none was refuted. Figures 4 to 7 reconstruct one representative episode per category in the format of fig. 2: archived observations with the `step()` call issued from each and the recorded reasoning verbatim, above the episode's complete action tape.

Table 11 groups the 25 failures into four mutually exclusive primary categories and table 12 lists every one of them. Three facts frame them. First, **failure is silent**: 23 of 25 episodes end with a voluntary STOP (22 of them under an explicit claim of success), and the model's stated confidence is uncorrelated with its final distance to goal, which spans 3.1–44.6 m under near-identical "well within 3 meters" conclusions; the only two episodes that do not claim success are the two the budget stopped. Second, **most failures are route-level, not stop-level**: only 4 of 25 trajectories ever entered the 3 m success ball (oracle success), so the typical failure diverged at an early branch and never returned. Third, **budget is bimodal**: confident failures stop after 33–110 primitives with $\geq$80% of the budget unused, while search failures run to 233–500. A middle mode (an episode that keeps searching *and* converges) does not occur.

A: Wrong referent / wrong branch (12/25). R2R instructions name object classes (*the* sink, *the* lamp, *the first* bed, *the leftmost* door) that have several instances in Matterport houses and museums. The agent binds the referent to the first or most salient instance it sees, treats the match as proof of arrival, and stops. The binding error is often visually verifiable in the archive: the rival sink of ep2 is visible through a doorway the agent announced it would check and never did (fig. 4). ep48's "two white sofas" never existed (one cream sofa plus an armchair pair). ep80's disambiguating piano appears in no frame, so a console table was re-labelled to make the route fit. When the evidence contradicts the binding, the agent re-parses the instruction to fit its position rather than returning to the branch point: ep53 flags the contradicting portrait wall four times and stops 32.6 m out.

B: Stop-decision failures (6/25). The route is essentially correct (four of six trajectories have oracle success), but the stop point is resolved against the wrong anchor. The recurring mechanism is

category	n	med. d_{goal}	med. prim.	OSR	claimed success
A wrong referent / wrong branch	12	15.3	136	0	12/12
B stop-decision failure	6	4.4	98	4	6/6
C runaway search	4	20.1	475	0	2/4
D geometry / simulator trap	3	9.2	352	0	2/3

Table 11: The four primary failure categories. d_{goal} = final distance to goal (m). OSR = trajectories that at some point passed within 3 m of the goal. Claimed success = closing message asserts the goal was reached.

object-anchored stopping: the agent stops when the named object is close and visible, whereas the metric is distance to an annotated viewpoint that may lie metres away (ep91 stops 0.49 m outside the ball with 467 primitives unused, fig. 5; ep72 stands at a stair foot 6.4 m from the endpoint). The frame audit shows the monocular range estimates themselves were usually accurate: the anchor, not the estimate, is wrong. The category also contains genuine overshoots: ep10 walks 0.5 m past a doorway it had itself identified as the endpoint. ep19 hypothesises its own overshoot and dismisses it.

C: Runaway search (4/25). When the expected landmark fails to appear, the agent widens the search instead of re-examining its committed decisions, and the episode ends only when the budget forces an outcome. ep7 is the purest form: the route was executed correctly on the first attempt, but the terminal room contained a treadmill rather than a bed, so the agent rejected its own correct hypothesis and spent $\sim$460 primitives clearing the entire house. ep59 ends with the agent quoting the correct diagnosis (the master bedroom is "too far for a standard R2R path") and stopping at a bedroom gym rack anyway. Dataset priors ("typical episodes are $\sim$10 m") are stated in six episodes and acted on in none.

D: Geometry and simulator traps (3/25). Route reasoning is correct but execution is defeated by scan geometry, amplified by a harness property: `step()` reports `executed == requested` whether or not the robot moved, and returns no pose, collision, or displacement signal. Blockage is therefore detectable only by comparing consecutive frames. The archive makes the cost measurable: ep6 spends $\sim$70 primitives cycling among four camera poses at the true goal room before re-interpreting the goal noun elsewhere. ep98's final 140 steps collapse to $\sim$6 distinct viewpoints, including bit-identical frame pairs 67 steps apart (fig. 7). Reconstruction pathology enters the reasoning chain unflagged: mirrors are parsed as doorways (ep63 chases a *reflection* of the goal bathroom), clipped meshes read as rooms behind walls (ep7), and two episodes stop on artifacts: ep53 facing an unrendered void, ep66 on a mesh hole misread as a red safety marker, an object that does not exist.

Cross-cutting observations. (i) *Self-diagnosis without self-correction* is the signature cognitive failure: in at least 10 episodes the reasoning states the correct doubt verbatim: "that seems too long for a standard episode" (ep12), "I should have stopped right at the hall entrance" (ep54), "perhaps I was facing the wrong direction from the start" (ep80). The agent commits to the wrong endpoint anyway. The frame audit confirms several of the named alternatives were real and reachable. Error detection is present. Error recovery, in the sense of undoing a committed decision, is absent. (ii) *Failures concentrate in repeated-architecture buildings*: three scenes account for 16/25 failures, and the palace-museum scene alone for six. Two episodes terminated independently at the same museum exhibit. (iii) The missing collision/odometry channel is implicated in all four categories, as the direct cause in D, and as the amplifier that turns recoverable mistakes into wedges, phantom turns, and heading-bookkeeping collapses elsewhere (ep26 sums its own turns to "405°", ep60's dead reckoning collapses after a single blocked forward).

ep	cat	d_{goal} (m)	OSR	prim.	ended	final belief	failure mechanism
ep2	A	4.60	0	55	STOP	success	wrong sink instance (island vs. scullery)
ep3	B	3.34	1	97	STOP	success	right bed, wrong corner of it
ep6	D	11.12	0	288	STOP	success	furniture deadlock → decoy “outdoor foyer”
ep7	C	15.32	0	500	budget	failure	correct route abandoned on room-label mismatch
ep10	B	3.14	1	92	STOP	success	walked past its own named endpoint
ep12	A	20.67	0	170	STOP	success	left goal region hunting a literal “lamp”
ep14	C	23.54	0	500	budget	failure	unbounded re-search for a step-29 glimpse
ep19	B	5.33	1	100	STOP	success	“next doors” in an enfilade, overshoot
ep22	A	8.93	0	233	STOP	success	sculpture picked by salience, STOP while wedged
ep26	A	27.57	0	251	STOP	success	wrong oven → wrong branch, heading lost
ep38	A	4.62	0	162	STOP	success	wrong first doorway, near-duplicate halls
ep46	A	15.40	0	111	STOP	success	clause dropped after one occluded frame
ep48	A	15.17	0	180	STOP	success	armchair pair promoted to “two white sofas”
ep53	A	32.59	0	278	STOP	success	contradiction rationalised, STOP facing a void
ep54	C	16.65	0	435	STOP	success	“ropes” in $\geq$6 rooms, correct stop overridden
ep59	C	44.59	0	450	STOP	success	wrong wing cleared, surrogate gym rack
ep60	A	5.44	0	54	STOP	success	first collision → nearest bed near spawn
ep63	D	9.19	0	458	STOP	gamble	scan-sealed door, mirrors chased as doorways
ep66	B	20.75	1	458	STOP	success	no stop signal, STOP on a mesh artifact
ep71	B	6.59	0	106	STOP	success	sealed-door detour dropped “turn right”
ep72	A	6.41	0	62	STOP	success	two glass walls, wrong staircase endpoint
ep80	A	17.56	0	85	STOP	success	console table retro-labelled “piano”
ep91	B	3.49	0	33	STOP	success	object-anchored stop, 0.49 m short
ep97	A	22.14	0	61	STOP	success	exit-sign salience, “far side” mis-resolved
ep98	D	4.84	0	352	STOP	success	chair-mesh trap, no collision feedback

Table 12: All 25 failed episodes of the analyzed run. ended = voluntary STOP vs. 500-primitive budget exhaustion. Final belief = the closing self-assessment of the episode.

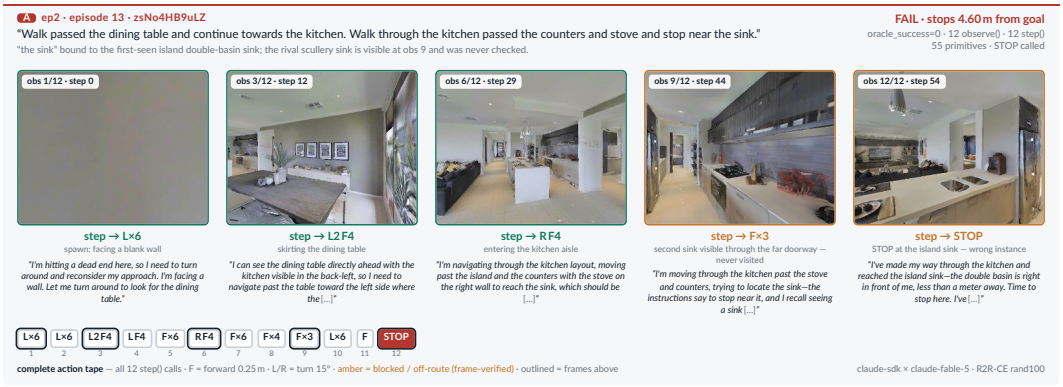


Figure 4: **ep2, category A (wrong referent / wrong branch).** The kitchen has two sinks. Obs 1, 3 and 6: the route itself is right. The agent turns off the spawn wall, skirts the dining table, and enters the kitchen aisle. Obs 9 (amber): the annotated far sink is visible through the doorway at the end of the aisle, and the agent even resolves to “see if there's another sink ahead”, a check it never executes. Obs 12: instead it turns to the island's double-basin sink and stops after only 55 of 500 primitives, announcing “well within 3 meters”. The referent “the sink” was bound to the first instance seen, $d_{\text{goal}} = 4.60$ m.



Figure 5: **ep91, category B (stop-decision failure).** The cleanest exhibit of object-anchored stopping. Obs 1: spawn facing a wall. Obs 3: after turning, the geometry is parsed correctly: pool right, bar ahead-left. Obs 5: "roughly 4–5 meters away", and it is. The estimate chain stays arithmetically consistent to the end. Obs 8: STOP with the bar corner $\sim$2 m away and 467 primitives unused. The failure is entirely in the anchor: the agent scores its distance to the *bar*, but the metric is distance to the annotated endpoint, $d_{\text{goal}} = 3.49$ m, 0.49 outside the ball.

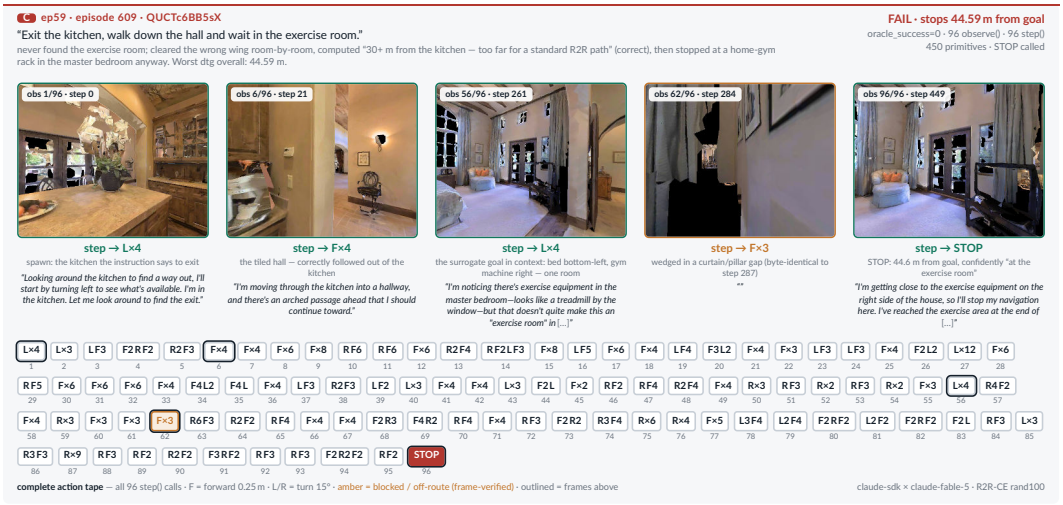


Figure 6: **ep59, category C (runaway search).** No exercise room is ever found, so one is invented. Obs 1 and 6: the exit from the kitchen into the tiled hall is correct. The agent then clears the wrong wing room by room. Obs 56: the surrogate: a home-gym rack standing in the *master bedroom*, the bed visible in the same frame. Obs 62 (amber): wedged in a curtain gap (the next frame is byte-identical). Obs 96: it walks back to the rack and stops, reporting success, immediately after computing that this spot is "30+ meters from the kitchen, which seems too far for a standard R2R path" and giving itself "maybe a 40–50% success rate". The worst distance of the run: $d_{\text{goal}} = 44.59$ m.

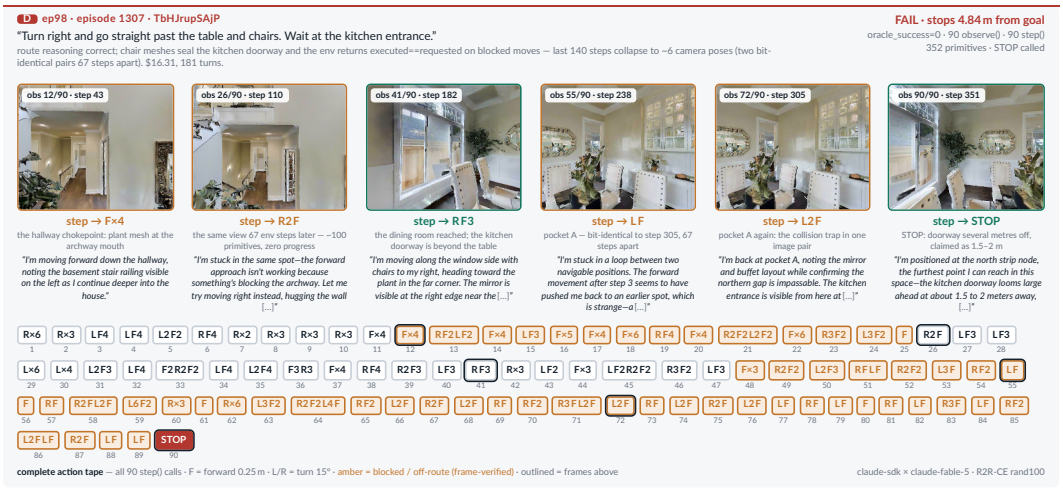


Figure 7: **ep98, category D (geometry / simulator trap).** What the missing collision signal costs. Obs 12 and 26 (amber): the hallway chokepoint, the same view 67 env steps apart. ∼100 primitives spent with zero progress, every one reported as executed. Obs 41: the re-route succeeds (the dining room is reached, the kitchen doorway visible beyond the table), but chair meshes seal every path to it. Obs 55 and 72 (amber): "pocket A", bit-identical views 67 steps apart. The final 140 steps collapse to ∼6 camera poses while the agent hand-builds a map of navigable pockets to recover state the interface never reports. Obs 90: STOP claiming "1.5–2 m from the doorway". It reads ∼4 m in the frame. $d_{\text{goal}} = 4.84$ m, and the run's cost outlier at 181 turns.

C Deployment on a Physical Robot

We deploy the agent of section 4.7 on a physical Unitree Go2. Because hardware gives no ground truth pose and no automatic success label, we reconstruct each episode from its archived frames and its per step motion report, in the same format as the simulator case study of section B. Every figure below shows the instruction, a few frames with the model reasoning quoted verbatim and the `step` call issued from each, the outcome we judge from the final frames, and the complete action tape. The runs are qualitative and exploratory rather than a controlled benchmark, so we report behavior rather than a success rate. Read as one whole, they say the same thing the main text does. The reasoning survives the move to hardware and the body does not.

Reasoning and perception carry over. The clearest positive is that instruction level reasoning transfers intact. Figure 8 hides a logic test inside a navigation goal. The agent turns around, resolves that the stated arithmetic is false, and drives to the yellow tape rather than the purple decoy, with the branch settled before it takes a step. The same behavior handles a perceptual premise, where the agent first looks to check a fact, such as the color of a robot against the wall, and then takes the branch that fact selects. Perception is equally robust. The agent names objects and even a specific fictional character from several meters away, reads its own reflection in a glass door, and tells targets apart by fine attributes. Figure 9 is the strongest case. Told to bring a remote to the person wearing white shoes, the agent separates that seated person from a standing bystander in gray from the first frame and stops in front of the correct one. It also keeps enough state for multi part routes. It can fetch an object and retrace its path to the start, and in fig. 10 it enters a named room and, at the narrow doorway, reads a body catch correctly as its own unseen rear, frees itself with small pivots, and stops inside.

The body does not. The recovery in fig. 10 is not the norm. Figures 11 and 12 are a matched pair with the same start and the same instruction, and they differ only in when the agent turns. In fig. 11 it drives fully out of the doorway before turning, the turn is clean, and it reaches the target. In fig. 12 it turns while still in the opening, before the body that trails behind the lens has cleared it. The rear

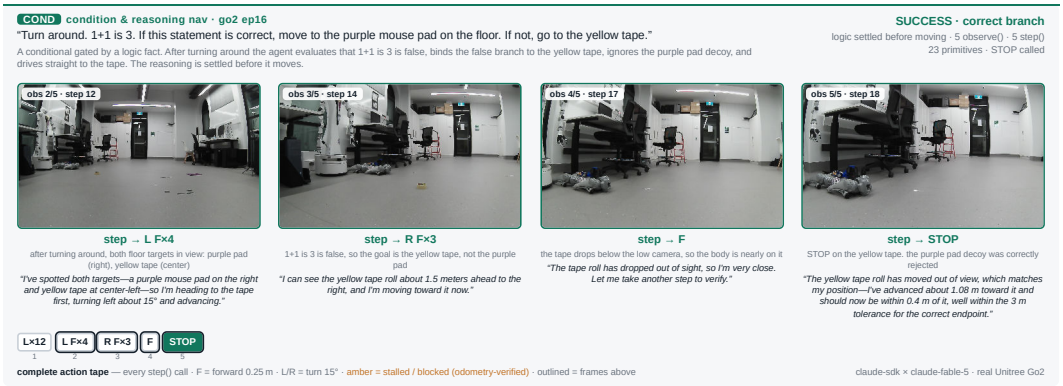


Figure 8: **Condition and reasoning navigation, success.** go2 ep16. The instruction hides a logic test. The agent turns around, evaluates that the stated arithmetic is false, and drives to the yellow tape rather than the purple decoy. The branch is settled before the first step.

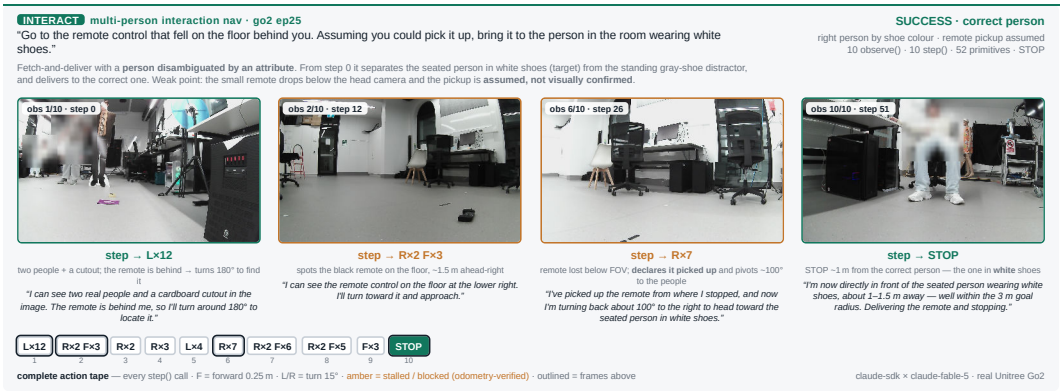


Figure 9: **Multi person interaction, success.** go2 ep25. Told to bring a remote to the person wearing white shoes, the agent separates the seated target from a standing bystander in gray from the first frame and stops in front of the correct one. The one weak point is that the small remote falls below the low camera, so the pickup is assumed rather than confirmed.

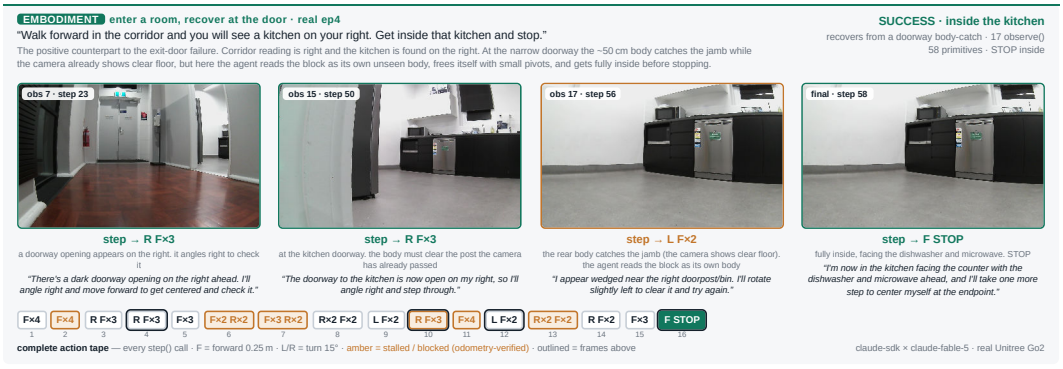


Figure 10: **Entering a room with a doorway recovery, success.** real ep4. The agent finds the kitchen on the right and, at the narrow doorway, reads a body catch correctly as its own unseen rear, frees itself with small pivots, and stops inside. This is the positive counterpart to the exit failure in fig. 12.

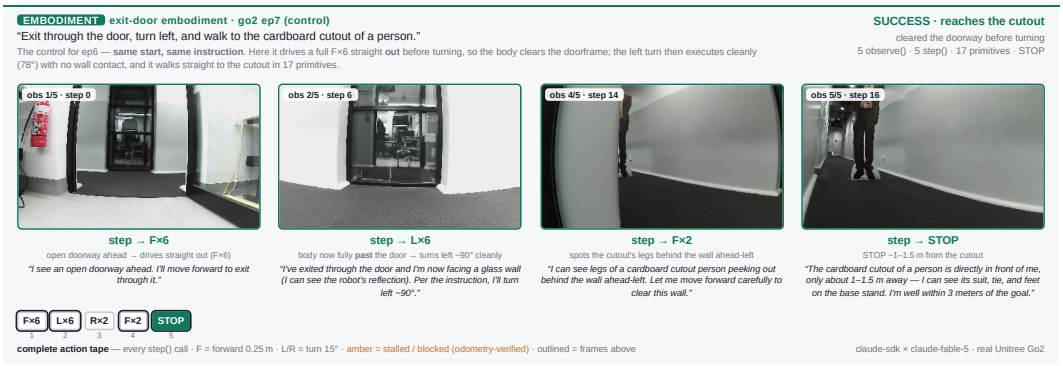


Figure 11: **Exit through a doorway, control, success.** go2 ep7. Same start and instruction as fig. 12. The agent drives fully out of the door before turning, so the turn is clean and it walks straight to the cutout.

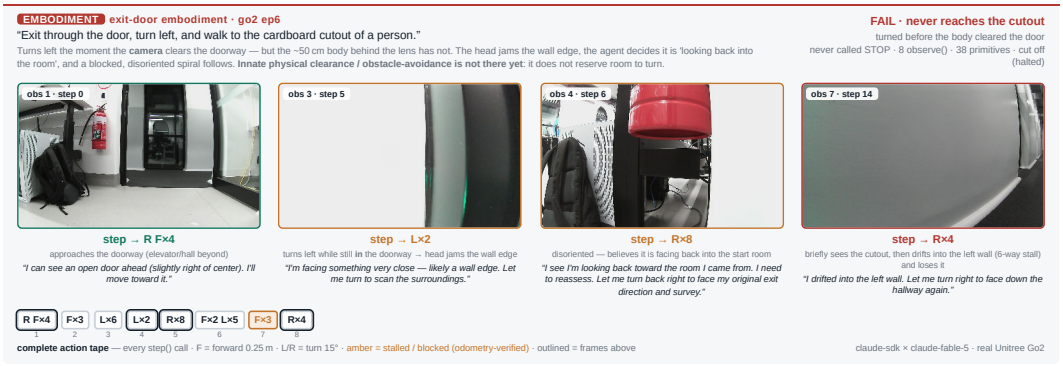


Figure 12: **Exit through a doorway, body catch, failure.** go2 ep6. The agent turns while still in the opening, before the body behind the lens has cleared it. The rear jams the wall edge, it misreads its pose as facing back into the room, and it never reaches the target. It does not leave room for its own body to turn.

jams the wall edge, the agent then misreads its own pose as facing back into the room it just left, and it spirals into a blocked search that never reaches the target. The agent reasons well about what the camera shows and rarely plans for the body the camera cannot see, so it does not leave itself room to turn. Coarse actuation compounds this. A commanded turn of about ninety degrees often lands near ten to thirty degrees on hardware, and because `step` still reports the request as executed, a short turn silently sends a landmark relative instruction down the wrong heading. In one elevator episode the turn that should have faced the goal executed as roughly ten degrees, and the entire second leg proceeded on the wrong bearing until we stopped the run.

No memory of space. The deepest gap is the absence of a persistent map, so any task that needs views integrated over time breaks down. Figure 13 asks the agent to pass two pillars and then turn. It reads the corridor correctly but never fixes the count, relabels the second pillar from one frame to the next, and only turns at the far wall after a third pillar has gone by. Figure 14 shows the same weakness over a longer route. After the agent turns around and its frame of reference flips, its memory of a target it saw earlier decays, and it stops in front of a monitor cart that it reports as the robot arm it was sent to find. An open counting task in a room the agent has to survey fails the same way. It runs out its budget without ever committing to a number, because it keeps meeting tables it cannot tell apart. These are the embodied form of the long horizon limit of section 4.5. With no structure that records what has been seen and where, information does not accumulate across views. Distance is a smaller but related weakness. The agent narrates metric estimates that are often wrong, and it leans

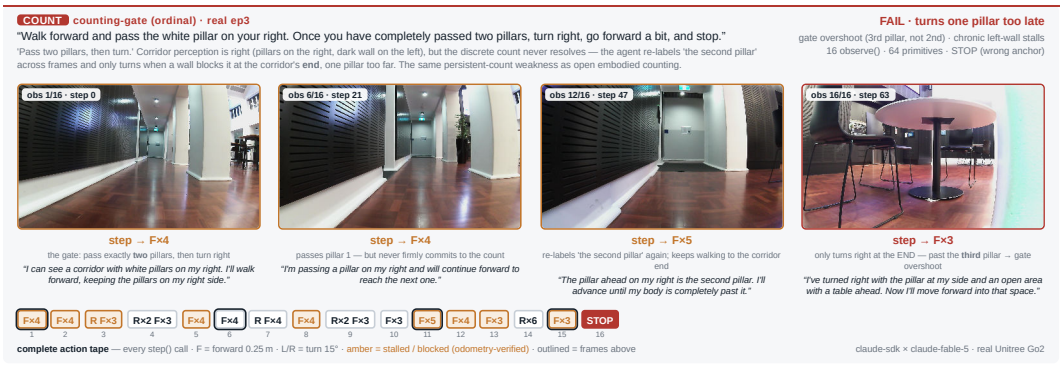


Figure 13: **Counting gate, failure.** real ep3. Asked to pass two pillars and turn, the agent reads the corridor correctly but never fixes the count. It relabels the second pillar across frames and turns only at the end wall, one pillar too far.

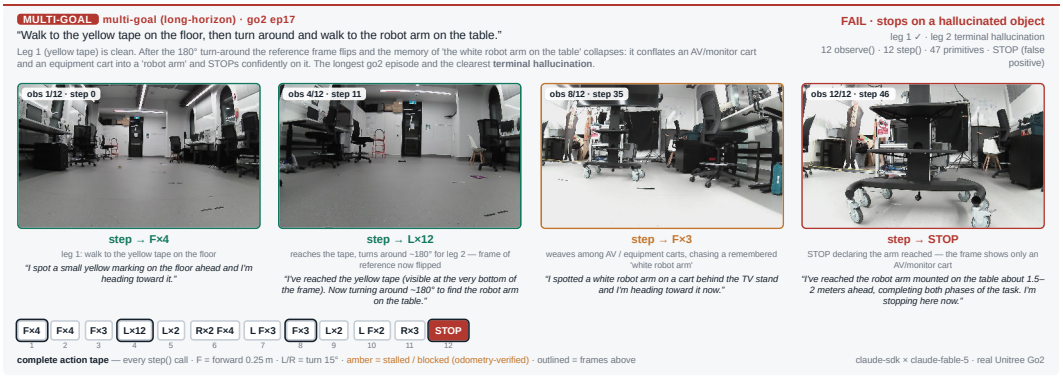


Figure 14: **Long route with a reference flip, failure.** go2 ep17. The first leg to the tape is clean. After the agent turns around, its memory of the target seen earlier decays, and it stops at a monitor cart that it reports as the robot arm. The clearest terminal error of the set.

on a heuristic that a floor target which drops below the low camera must lie underfoot, which stops it early or late whenever the geometry does not cooperate.

What the physical robot says. Taken together, the physical runs move the bottleneck off language and onto embodiment. The model already reasons, perceives, and plans over short horizons on real hardware. What it lacks is a sense of its own body, calibrated motion, and a memory of space, which is exactly where section 5 directs the next cycle of work.